Dynamic memory allocation for over-aligned data

Doc. No.:	P0035R0
Revises:	N3396
Date:	2015-09-09
Reply to:	Clark Nelson
Phone:	+1-503-712-8433
Email:	clark.nelson@intel.com

Problem statement

To codify widespread existing practice, C++11 added the ability to specify increased alignment (a.k.a. over-alignment) for class types. Unfortunately (but also consistently with existing practice), C++11 did not specify any mechanism by which over-aligned data can be dynamically allocated correctly (i.e. respecting the alignment of the data). For example:

```
class alignas(16) float4 {
        float f[4];
};
float4 *p = new float4[1000];
```

In this example, not only is an implementation of C++11 **not** required to allocate properly-aligned memory for the array, for practical purposes it is very nearly required to do the allocation incorrectly. In any event, it is certainly required to perform the allocation by a process that does **not** take the specified alignment value into account.

This represents a hole in the support for alignment in the language, which really needs to be filled.

History

With the exception of this section, and the “Nitty-gritty” section below, this document is virtually identical to N3396, which was discussed by EWG at the 2012 Portland meeting.

Since that time, Intel has released a compiler that largely implements the language changes discussed herein, except that, to guarantee backward compatibility, the additional overloads are declared in a new header (`<aligned_new>`), instead of being predeclared or declared in `<new>`.

To date, there has not yet been enough experience with the implementation to prove its viability. However, it seems appropriate to get this issue back on the committee's radar, so that a decision can be made about it for the C++17 time frame.

Design considerations

Backward compatibility

One of the first questions that needs to be settled about the future direction is the degree to which backward compatibility with C++11 needs to be maintained. On the one hand, in an ideal world, for an example like the one above, it would be obvious that the specified alignment should be honored.

On the other hand, there's no way to achieve that ideal without at least potentially changing the behavior of some C++11 programs. For example, a program might asssume control of dynamic allocation through the use of class-specific `operator new` and `operator delete` functions, or by replacing the global functions. These functions don't take any alignment argument. If a different function is used instead, which is somehow passed an alignment value, some degree of backward compatibility is lost.

When backward compatibility and the ideal future direction are in conflict, which should take precedence, and to what degree?

If perfect backward compatibility with C++11 were required, one way to ensure that might be to require that a new header — say `<aligned_new>` — be included in order to get new dynamic allocation for over-aligned types. But that would sacrifice convenience and/or correctness; using `alignas` by itself would presumably never be enough to get correctly aligned dynamic allocation.

Another obvious position to take would be that backward compatibility with C++98, which had no alignment specifier, needs to be complete. This might suggest that dynamic allocation should differ between types involving alignment specifiers and types that don't — which some might consider to be an unfortunate complication.

In C++11, when an over-aligned class type has its own dynamic memory allocation functions, it would be reasonable to hope that those functions already do the right thing with respect to alignment, and dangerous to make any change. However, the only way over-alignment could be accommodated by global allocation and deallocation functions would be to replace them with functions that always provide the strictest alignment used by any type in the program. It may be reasonable to assume that very few programs go to that length, instead of using class-specific allocation/deallocation.

Therefore, it may be acceptable to abandon backward compatibility with C++11 with respect to calling a global allocation function for dynamic allocation of an over-aligned type. But if so, that may well be the only acceptable case.

Passing the alignment value

To minimize the possibility of conflict with existing placement allocation functions, it might be advisable to invent a new standard enumeration type to use for alignment parameters; for example:

```
namespace std {
        enum class align_val_t: size_t;
};
void *operator new(std::size_t, std::align_val_t);      // new overload
```

It's not clear that this type would need any named constants of its own; it just needs to be able to represent alignment values, which are associated with type `size_t`. It should perhaps nevertheless be a scoped enumeration, to prevent the possibility that a value of that type would inadvertently be converted to some integer type, and match an existing placement allocation function.

If an allocation function that takes an alignment value is available, it should be used, for the sake of generality; but if no such function is available, a function that doesn't take one should be used, for backward compatibility. This suggests a new rule for new-expressions: attempting to find an allocation function in two phases, with two different sets of arguments.

Class-specific allocation and deallocation

It should be kept in mind that, under the current language rules, any class-specific allocation functions effectively hide all global allocation functions, including the ones in the standard library. For example, the following is invalid:

```
#include <new>
class X {
        void *operator new(size_t);     // no operator new(size_t, std::nothrow_t)
        void operator delete(void *);
};
X *p = new(nothrow) X;  // ::operator new(size_t, std::nothrow_t) is not considered
```

It is possible to imagine adjusting the rules to enable finding an alignment-aware allocation function more often, but that would also make it more likely that some programmers would write programs believing — incorrectly — that they have taken over complete control of the way that their class is dynamically allocated.

Unified vs. distinct arenas

What implementation techniques should the standard allow for allocation and deallocation of aligned memory?

In POSIX, there is a function named `posix_memalign` that can allocate over-aligned memory; `free` is used to free the blocks it allocates.

On Windows, on the other hand, of course `malloc`, `realloc` and `free` are supported for default-aligned memory. In addition, for over-aligned memory, there are functions named `_aligned_malloc`, `_aligned_realloc`, and `_aligned_free`. Memory that's allocated by `_aligned_malloc` must be freed by `_aligned_free`, and memory that's allocated by `malloc` must be freed by `free`. So logically, there are two disjoint, non-interoperable memory arenas; the program has to know to which arena a block belongs (i.e. how it was allocated) in order to be able to free it.

This is almost certain to be true of any implementation where over-aligned memory allocation is layered on top of “plain old” default-aligned memory allocation. There are probably many such implementations, and they're not likely to go away soon.

In an environment where information about the method used to allocate a block of memory can be lost, having distinct arenas (i.e. distinct deallocation functions) could be inconvenient. A program whose operation depends on the assumption that `operator new` is equivalent to `malloc` is effectively an environment where information about the method used to allocate a block of memory is lost.

But in a well-written, portable C++ program, at the point where memory is deallocated, the type of the object being deleted — and therefore whether it is over-aligned — is known. This knowledge could, and probably should, be used to support layered implementations of over-aligned memory allocation.

This implies that, just as a new-expression for an over-aligned type should look for an alignment-aware allocation function, so should a delete-expression for a pointer to an over-aligned type look for an alignment-aware deallocation function. Presumably this would be done by selecting a deallocation function to which the alignment value can be passed, even though probably very few implementations will actually have any use for that value.

Nitty-gritty

For exactly what classes should the allocation method change? Plausible answers include:

1. Those affected by an `alignas` that actually specifies over-alignment.
2. Those affected by an explicit `alignas`, even if the alignment value is basic (i.e. small).

The first answer seems to be right from a pragmatic perspective, but one consequence is that the behavior of a program might depend (in a new way) on an implementation-defined parameter. If the only difference between alignment-aware and alignment-unaware allocation/deallocation functions is the actual allocation mechanism (i.e., in a well-designed program), this should not be a problem. It's rather like the implementation's license to elide certain copies, which implies that a copy constructor had really better just make a copy.

The below WD changes use the first answer, through use of “over-aligned”. The Intel implementation uses the first answer by default, but has a command-line option to select the second answer, for the sake of experimentation.

Assuming the existence of a variety of allocation functions, which one should be used for an over-aligned allocation? I believe the answer should be the first one from the following list that is known to exist:

1. class-specific and alignment-aware
2. class-specific and alignment-unaware
3. global and alignment-aware
4. [global and alignment-unaware]

Here “alignment-aware” means “having an explicit alignment parameter”.

A class-specific, alignment-unaware allocation function is preferred over one that is global and alignment-aware because there are many cases where a class-specific allocation function has enough information, even without an explicit parameter, to do the allocation with sufficient alignment. (Likely exceptions include a template class with a base or member of a type that is a template parameter, and a derived class that inherits its allocation function from a base class, and also adds a member or base of over-aligned type.)

If a global, alignment-aware allocation function is predeclared, then it will never be necessary to use a global, alignment-unaware allocation function for an over-aligned type; hence the brackets around item 4.

Outline of possible working paper changes

The following changes are intended to be suggestive, not definitive. They are definitely incomplete, but they give a sense of the flavor and some idea of the scope of the form I believe the changes will eventually take. The particularly important changes are presented first.

Mainly for simplicity, here I suggest that the new overloads should be added to `<new>`, and for consistency with that, that they should also be predeclared. But if 100% backward compatibility with C++11 is considered necessary, then the new overloads probably need to be declared in a new library header (possibly `<aligned_new>`). It's also possible to imagine requiring the declarations be in a new header, but making it implementation-defined whether that header is included by `<new>`, perhaps with the expectation that the actual choice will be left to users, under the control of a command-line option or macro setting.

There is one change of terminology worth noting. Today, the phrase “placement new” is ambiguous. In some contexts it means adding arguments to a call to an allocation function, with any types and unspecified purpose. In other contexts, it is used to refer specifically to cases where there is a single additional argument of type `void *`, in which case the allocation function doesn't actually allocate anything. I refer to the latter cases as “non-allocating”, and refer to “allocating” cases to distinguish them when necessary.

Change 18.6, header `<new>` synopsis:

```
namespace std {
        class bad_alloc;
        class bad_array_new_length;
        enum class align_val_t: size_t;
        struct nothrow_t {};
        extern const nothrow_t nothrow;
        typedef void (*new_handler)();
        new_handler get_new_handler() noexcept;
        new_handler set_new_handler(new_handler new_p) noexcept;
};
void* operator new(std::size_t size);
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void* operator new[](std::size_t size);
```

```
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;

void* operator new(std::size_t size, std::align_val_t alignment);
void* operator new(std::size_t size, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void* operator new[](std::size_t size, std::align_val_t alignment);
void* operator new[](std::size_t size, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;

void* operator new (std::size_t size, void* ptr) noexcept;
void* operator new[](std::size_t size, void* ptr) noexcept;
void operator delete (void* ptr, void*) noexcept;
void operator delete[](void* ptr, void*) noexcept;
```

Change 5.3.4p11:

> The *new-placement* syntax ~~is~~ can be used to supply additional arguments to an allocation function. ~~If used, overload~~ Overload resolution is performed on a function call created by assembling an argument ~~list consisting of~~ list. The first argument is the amount of space requested ~~(the first argument)~~ , and has type `std::size_t`. If the type of the allocated object is over-aligned, the next argument is the type's alignment, and has type `std::align_val_t`. ~~and the~~ If the *new-placement* syntax is used, its expressions ~~in the *new-placement* part of the *new-expression* (~~ are the ~~second and~~ succeeding arguments~~)~~. ~~The first of these arguments has type `std::size_t` and the remaining arguments have the corresponding types of the expressions in the *new-placement*.~~ If no matching function is found and the allocated object type is over-aligned, the alignment argument is removed from the argument list, and overload resolution is performed again.

Change 5.3.4p12:

> [*Example:*
>
> - `new T` results in a call of either `operator new(sizeof(T), static_cast<std::align_val_t>(alignof(T)))` or `operator new(sizeof(T))`,
> - `new(2,f) T` results in a call of either `operator new(sizeof(T), static_cast<std::align_val_t>(alignof(T)),2,f)` or `operator new(sizeof(T),2,f)`,
> - `new T[5]` results in a call of either `operator new[](sizeof(T)*5+x, static_cast<std::align_val_t>(alignof(T)))` or `operator new[](sizeof(T)*5+x)`, and
> - `new(2,f) T[5]` results in a call of either `operator new[](sizeof(T)*5+y, static_cast<std::align_val_t>(alignof(T)),2,f)` or `operator new[](sizeof(T)*5+y,2,f)`.
>
> ...

Most of the rest of these changes are just reflecting the implications of the above changes through the rest of the document.

If the new overloads should be predeclared, change 3.7.4p2:

> The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (18.6.1). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library (17.6.4.6). The following allocation and deallocation functions (18.6) are implicitly declared in global scope in each translation unit of a program.

```
void* operator new(std::size_t);
void* operator new[](std::size_t);
void operator delete(void*);
void operator delete[](void*);

void* operator new(std::size_t, std::align_val_t);
void* operator new[](std::size_t, std::align_val_t);
void operator delete(void*, std::align_val_t);
void operator delete[](void*, std::align_val_t);
```

> These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`. [*Note:* The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression* or function

call that refers to one of these functions without including the header `<new>` is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` is ill-formed unless the name has been declared by including the appropriate header. —*end note*] Allocation and/or deallocation functions can also be declared and defined for any class (12.5).

Change 3.7.4.2p3:

If a deallocation function terminates by throwing an exception, the behavior is undefined. The value of the first argument supplied to a deallocation function may be a null pointer value; if so, and if the deallocation function is one supplied in the standard library, the call has no effect. Otherwise, the behavior is undefined if the value supplied to `operator delete(void*)` in the standard library is not one of the values returned by a previous invocation of ~~either `operator new(std::size_t)` or `operator new(std::size_t, const std::nothrow_t&)`~~ an allocating form of `operator new` in the standard library, and the behavior is undefined if the value supplied to `operator delete[](void*)` in the standard library is not one of the values returned by a previous invocation of ~~either `operator new[](std::size_t)` or `operator new[](std::size_t, const std::nothrow_t&)`~~ an allocating form of `operator new[]` in the standard library.

Change 3.7.4.3p2:

A pointer value is a *safely-derived pointer* to a dynamic object only if it has an object pointer type and it is one of the following:

- the value returned by a call to the C++ standard library implementation of ~~`::operator new(std:: size_t)`~~ any of the allocating forms of `operator new` or `operator new[]`;[37]
- ...

Change 17.6.4.6p2:

A C++ program may provide the definition for any of ~~eight~~ sixteen dynamic memory allocation function signatures declared in header `<new>` (3.7.4, 18.6):

- `operator new(std::size_t)`
- `operator new(std::size_t, const std::nothrow_t&)`
- `operator new[](std::size_t)`
- `operator new[](std::size_t, const std::nothrow_t&)`
- `operator delete(void*)`
- `operator delete(void*, const std::nothrow_t&)`
- `operator delete[](void*)`
- `operator delete[](void*, const std::nothrow_t&)`

- `operator new(std::size_t, std::align_val_t)`
- `operator new(std::size_t, std::align_val_t, const std::nothrow_t&)`
- `operator new[](std::size_t, std::align_val_t)`
- `operator new[](std::size_t, std::align_val_t, const std::nothrow_t&)`
- `operator delete(void*, std::align_val_t)`
- `operator delete(void*, std::align_val_t, const std::nothrow_t&)`
- `operator delete[](void*, std::align_val_t)`
- `operator delete[](void*, std::align_val_t, const std::nothrow_t&)`

Add descriptions of the new functions to sections 18.6.1.1 and 18.6.1.2.

Change 18.6.1.1p12:

Requires: ptr shall be a null pointer or its value shall be a value returned by an earlier call to ~~the~~ an allocating form of `operator new` (possibly replaced) ~~`operator new(std::size_t)` or `operator new(std::size_t,const std::nothrow_t&)`~~ which has not been invalidated by an intervening call to `operator delete`~~`(void*)`~~.

Change 18.6.1.2p11:

Requires: ptr shall be a null pointer or its value shall be the value returned by an earlier call to an allocating form of `operator new[]`~~`(std::size_t)` or `operator new[](std::size_t,const std::nothrow_t&)`~~ which has not been invalidated by an intervening call to `operator delete[]`~~`(void*)`~~.

Change the title of section 18.6.1.3:

18.6.1.3 ~~Placement~~ Non-allocating forms **[new.delete.placement]**

Change 20.6.9.1p6:

Remark: the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called. The use of `hint` is unspecified, but intended as an aid to locality if an implementation so desires.